

CS-471L: Compiler Construction Lab

Symbol Table Manager

Spring 2026

University of Engineering and Technology, Lahore

Department of Computer Science

1. Objectives

By the end of this lab, the student will be able to:

- Explain the role of the symbol table inside a compiler.
- Choose a suitable data structure for the symbol table.
- Implement insert, lookup, and delete on a hash-based symbol table.
- Handle nested scopes using a stack of symbol tables.
- Detect duplicate declarations and use of undeclared names.
- Connect the symbol table with the parser built in previous labs
- Print the contents of the symbol table for debugging.

2. Background Theory

2.1 What is a Symbol Table?

The symbol table is a data structure that the compiler keeps to remember information about every identifier (variable, constant, function, type, parameter, array, class) used in the source program. Whenever the compiler sees a declaration, it inserts an entry. Whenever it sees a use of a name, it looks up the entry to fetch the type and other attributes.

Without a symbol table, the compiler cannot perform scope checking, type checking, or memory allocation. The symbol table is used by almost every phase that comes after the lexer.

2.2 Information Stored in an Entry

A typical symbol table entry stores the following attributes:

- Name (the lexeme of the identifier).
- Kind (variable, constant, function, parameter, array, class).
- Type (int, char, real, bool, void, or a user-defined type).
- Scope level (the depth or id of the enclosing scope).
- Source line number where the name was declared.
- Memory offset (used later for code generation).
- Extra attributes such as array size, function parameter list, return type.

2.3 Choice of Data Structure

Several data structures can hold the symbol table. Their trade-offs are summarised below.

Structure	Insert	Lookup	Notes
Linear list	$O(1)$	$O(n)$	Simple to write, slow for large programs.

Sorted list	$O(n)$	$O(\log n)$	Binary search lookup but slow insert.
Binary search tree	$O(\log n)$ avg	$O(\log n)$ avg	Worst case $O(n)$ if unbalanced.
Hash table	$O(1)$ avg	$O(1)$ avg	Recommended for this lab. Use chaining for collisions.

In this lab the symbol table is built using a hash table with separate chaining.

2.4 Hash Function

A simple hash function for strings is the sum of ASCII codes modulo the table size:

```
h(s) = (sum of ASCII codes of characters in s) mod TABLE_SIZE
```

A better and well-known hash for strings is djb2:

```
function hash(s):
    h = 5381
    for each character c in s:
        h = ((h << 5) + h) + c      /* h * 33 + c */
    return h mod TABLE_SIZE
```

A prime table size such as 211 or 1009 helps reduce clustering.

2.5 Scope Management

Most languages have block-structured scopes. The symbol table must handle nested scopes so that an inner declaration can hide an outer one with the same name. Two common designs are used:

- **Design A Stack of symbol tables:** one hash table per scope. When a new scope is entered, a fresh table is pushed on the scope stack. When the scope ends, the top table is popped (and usually printed). Lookup searches from the top of the stack downward.
- **Design B Single table with scope chain:** one global hash table. Each entry carries its scope id. Entries with the same name from different scopes are chained, with the most recent at the head. On end-of-scope, all entries with that scope id are removed.

Design A is used in this lab because it is easier to print and to debug.

2.6 Operations

- `insert(name, attributes)` add a new entry in the current scope. Return an error if the name is already declared in the current scope.
- `lookup(name)` search from the current scope outward. Return the most recent matching entry, or NULL if not found.
- `lookup_current(name)` search only the current scope. Used to detect duplicate declarations.
- `delete(name)` remove an entry from the current scope. Mainly used at end-of-scope.
- `beginScope()` push a new empty symbol table on the scope stack.
- `endScope()` pop the top symbol table, after printing its contents for debugging.
- `print()` print the contents of all the symbol tables in the scope stack.

3. Pre-Lab Requirements

Before attending this lab, every student must have:

- Completed Lab 3 (Lexical Analyzer) so that the tokenizer returns (token, lexeme, line, column) for each token.
- Completed Lab 4 (Recursive Descent Parser) for the assigned grammar.
- A copy of the assigned BNF grammar with at least variable and function declarations.
- Read Section 2.7 of the Dragon Book (symbol tables) and Chapter 7 §7.6 (scope handling).
- Brought a laptop with previous labs source code ready to extend.

4. Lab Tasks

The lab has five graded tasks. Tasks 1 and 2 must be completed during the 3-hour lab slot. Tasks 3, 4, and 5 are post-lab homework, due before the lab evaluation in Week 14.

Task	Description	Weight / When
Task 1	Build the basic hash table for a single scope: insert, lookup, delete, print.	25% · In-lab
Task 2	Add scope management: beginScope, endScope, lookup across the scope stack.	25% · In-lab
Task 3	Integrate with the parser. Detect duplicate declarations and undeclared names.	30% · Homework
Task 4	Pretty-print the symbol table in tabular form.	10% · Homework
Task 5	Short test report with test cases and observed output.	10% · Homework

4.1 Task 1 Basic Hash Table (In-Lab)

- Use a fixed-size array of slots. Default size 211 (a prime number).
- Use separate chaining for collisions (a linked list of entries per slot).
- **Implement:** `insert`, `lookup`, `delete`, `print`.
- Test the table with at least 10 inserts, 5 lookups (including some that miss), and 3 deletes.
- Do not yet worry about scopes. One table is enough at this stage.

4.2 Task 2 Scope Management (In-Lab)

- Wrap the hash table in a structure that also keeps a scope level and a parent pointer.
- **Implement:** `beginScope`, `endScope`, `lookup`, `lookup_current`.
- `lookup` must search from the current scope outward until it finds a match or reaches the global scope.
- `endScope` must print the contents of the popped scope before discarding it.
- Test with at least three nested scopes. Confirm that an inner declaration hides an outer declaration with the same name.

4.3 Task 3 Parser Integration (Homework)

Modify the parser from Lab 4 (or Lab 5) so that it calls the symbol table at the right points:

- On each variable / constant / function declaration, call insert. Report an error if the name is already declared in the current scope.
- On each use of a name in an expression or assignment, call lookup. Report an error if the name is not declared anywhere.
- On entry to a function body or a new block, call beginScope.
- On exit from a function body or a block, call endScope.

The compiler must detect at least the following two errors:

- Duplicate declaration in the same scope.
- Use of an undeclared variable.

4.4 Task 4 Pretty Print (Homework)

Print the symbol table in a clean tabular form, similar to:

ID	Name	Kind	Type	Scope	Line
1	main	function	int	0	1
2	x	variable	int	1	2
3	y	variable	int	1	3
4	z	variable	int	2	6

4.5 Task 5 Test Report (Homework)

Write a one to two page report that contains:

- At least five test cases (mix of valid and invalid programs).
- The expected output for each test case.
- The actual output produced by your compiler.
- Any limitations or known bugs.

5. Sample Pseudocode

The pseudocode below is a guide. Students may write the code in C, C++, Java, or Python. The implementation language must be the same one used in the earlier labs.

5.1 Entry and Table Structures

```

struct Entry {
    char *name;
    int kind;           /* 0=var, 1=const, 2=func, 3=array, 4=class */
    int type;          /* 0=int, 1=char, 2=real, 3=bool, 4=void */
    int scope_level;
    int line;
    struct Entry *next; /* chain in the same hash slot */
};

#define TABLE_SIZE 211

```

```

struct SymTable {
    struct Entry    *slots[TABLE_SIZE];
    int             scope_level;
    struct SymTable *parent;    /* link to the enclosing scope */
};

```

5.2 Hash Function

```

int hash(const char *s) {
    unsigned long h = 5381;
    for (int i = 0; s[i] != '\0'; i++)
        h = ((h << 5) + h) + (unsigned char) s[i];
    return h % TABLE_SIZE;
}

```

5.3 Insert

```

Entry* insert(SymTable *t, const char *name,
              int kind, int type, int line) {
    if (lookup_current(t, name) != NULL)
        return NULL; /* duplicate declaration */
    int h = hash(name);
    Entry *e = (Entry *) malloc(sizeof(Entry));
    e->name = strdup(name);
    e->kind = kind;
    e->type = type;
    e->scope_level = t->scope_level;
    e->line = line;
    e->next = t->slots[h];
    t->slots[h] = e;
    return e;
}

```

5.4 Lookup

```

Entry* lookup(SymTable *t, const char *name) {
    int h = hash(name);
    while (t != NULL) {
        for (Entry *e = t->slots[h]; e != NULL; e = e->next)
            if (strcmp(e->name, name) == 0) return e;
        t = t->parent; /* climb to enclosing scope */
    }
    return NULL; /* not declared */
}

Entry* lookup_current(SymTable *t, const char *name) {
    int h = hash(name);
    for (Entry *e = t->slots[h]; e != NULL; e = e->next)
        if (strcmp(e->name, name) == 0) return e;
    return NULL;
}

```

5.5 Begin and End Scope

```
SymTable* beginScope(SymTable *current) {
    SymTable *t = (SymTable *) calloc(1, sizeof(SymTable));
    t->parent      = current;
    t->scope_level = (current == NULL) ? 0 : current->scope_level + 1;
    return t;
}

SymTable* endScope(SymTable *current) {
    print_table(current);          /* dump before discard */
    /*
    SymTable *parent = current->parent;
    free_table(current);
    return parent;
    */
}
```

6. Sample Input and Expected Output

6.1 Sample Input (must be a source program in assigned language)

```
int main() {
    int x;
    int y;
    x = 5;
    {
        int z;
        z = x + y;
        int x;      /* shadows outer x */
        x = z;
    }
    int x;          /* ERROR: duplicate at line 11 */
    a = 1;          /* ERROR: undeclared 'a' at line 12 */
    return 0;
}
```

6.2 Expected Output

```
[Scope 0] Enter
[Scope 0] insert main : function, int, line 1
[Scope 1] Enter
[Scope 1] insert x      : variable, int, line 2
[Scope 1] insert y      : variable, int, line 3
[Scope 1] lookup x      -> found at scope 1
[Scope 2] Enter
[Scope 2] insert z      : variable, int, line 6
[Scope 2] lookup x      -> found at scope 1
[Scope 2] lookup y      -> found at scope 1
[Scope 2] insert x      : variable, int, line 8      /* shadows */
[Scope 2] Exit, dump:
    x : variable, int, line 8
    z : variable, int, line 6
ERROR line 11: duplicate declaration 'x'
ERROR line 12: undeclared variable 'a'
[Scope 1] Exit, dump:
    x : variable, int, line 2
```

```

    y : variable, int, line 3
[Scope 0] Exit, dump:
    main : function, int, line 1

```

7. Evaluation Criteria

The lab is graded out of 100 marks using the rubric below. Each task is evaluated separately. Marks may differ between students if individual contributions are unequal.

Component	Excellent (90–100%)	Good (70–89%)	Satisfactory (50–69%)	Poor (<50%)
Hash Table	Correct, efficient, well-tested	Correct for normal cases	Works for simple inputs	Major issues
Scope Handling	Stack of tables, shadowing works	Basic begin/end scope	Single scope only	Not functional
Parser Integration	Detects duplicates and undeclared	Detects one of the two errors	Calls table but no error checks	Not connected
Pretty Print	Clean tabular dump per scope	Readable output	Plain print	Hard to read
Test Report	5+ test cases, clear analysis	3–4 test cases	1–2 test cases	Missing or unclear
Viva	Confident, explains design choices	Answers most questions	Knows the code	Cannot explain

8. Deliverables

- Source code (well commented) of the symbol table module.
- Test driver that exercises insert, lookup, delete, beginScope, endScope.
- Sample input file and the corresponding output file.
- Short test report (one to two pages).
- README file with build and run instructions.

Note: the source code submitted for this lab will be reused in the final mini compiler project. Keep the module clean and well documented.

9. Submission Guidelines

File Naming

CS471L_Lab13_RollNo_Name.zip

Example: CS471L_Lab13_2021CS101_AhmadAli.zip

Folder Layout

submission/

— src/	(source files)
— test/	(sample input files)
— output/	(output for each test)
— report.pdf	(one to two page test report)
— Makefile	(build script)
— README.md	(build and run instructions)

Platform

Upload on the UET LMS before the deadline. Email submissions are not accepted.

Late Submission Policy

Days Late	Penalty
1 day	−10%
2 days	−20%
3 days	−30%
More than 3 days	Not accepted

10. Common Pitfalls

- Forgetting to call `lookup_current` before `insert`. This lets duplicate declarations slip through.
- Searching only the current scope on lookup. Names in enclosing scopes will appear undeclared.
- Calling `free` on the parent table at `endScope`. Only the top scope must be freed.
- Using a non-prime table size. This increases clustering.
- Forgetting to copy the lexeme with `strdup`. The lexer buffer is reused, so the pointer goes stale.
- Mixing scope levels with scope ids. Pick one and stick to it.

11. References

- Aho, Lam, Sethi, Ullman. Compilers: Principles, Techniques, and Tools (Dragon Book, 2nd Edition). Sections 2.7 and 7.6.
- Appel, A. W. Modern Compiler Implementation in C. Chapter 5.
- Lab manual notes uploaded on Google Classroom.

End of Lab Manual —